

(2025-2026-2)-033082Q1-算法设计与分析 题库

说明：题目描述均不重复；综合设计题均给出标准答案。

一、选择题

1. 快速排序一次划分后，基准元素通常满足什么位置关系？

- A. 位于数组末尾且不再比较 B. 左侧元素不大于它、右侧元素不小于它 C. 所有元素均已全局有序 D. 只保证相邻元素有序

答案：B

2. 归并排序中“归并”操作的主要任务是什么？

- A. 随机选择基准元素 B. 删除重复元素 C. 合并两个已有序子序列 D. 构造最小生成树

答案：C

3. 对已经按升序排列的序列执行直接插入排序，其典型时间复杂度是多少？

- A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(2^n)$

答案：A

4. 用数学归纳法证明递归算法正确性时，通常需要证明哪两部分？

- A. 排序与查找 B. 上界与下界 C. 入队与出队 D. 基础情形与归纳步骤

答案：D

5. 归纳证明中，假设命题对 $n=k$ 成立后，下一步通常要证明什么？

- A. $n=0$ 不成立 B. $n=k+1$ 成立 C. k 被删除 D. 所有输入随机成立

答案：B

6. 对递推式 $S(n)=S(n-1)+n$ 且 $S(1)=1$ ，常用归纳法证明的结果是哪一个？

- A. $S(n)=2n$ B. $S(n)=n^2$ C. $S(n)=n(n+1)/2$ D. $S(n)=\log n$

答案：C

7. n 皇后回溯算法中，通常按行放置皇后并检查什么冲突？

- A. 列和对角线 B. 队列长度 C. 哈希地址 D. 边权是否为负

答案：A

8. 回溯法发现当前部分解不可能得到可行解时，应采取什么操作？

- A. 继续扩展所有子结点 B. 改用顺序查找 C. 输出当前部分解 D. 剪枝并回退

答案：D

9. 子集和问题的回溯状态通常包含当前考察位置和什么信息？

- A. 树的高度 B. 队列容量 C. 当前已选元素和 D. 基准元素下标

答案：C

10. 动态规划适用的问题通常同时具有最优子结构和什么特征？

- A. 输入必须有序 B. 重叠子问题 C. 边权必须相同 D. 只能用递归实现

答案：B

11. LCS 动态规划中，当 $x[i-1] == y[j-1]$ 时， $dp[i][j]$ 应如何转移？

- A. $dp[i-1][j-1]+1$ B. $dp[i-1][j]$ C. $dp[i][j-1]$ D. 置为 0

答案：A

12. 0/1 背包一维动态规划中，容量通常按什么方向枚举以避免重复选取同一物品？

- A. 随机方向 B. 从小到大 C. 只枚举奇数容量 D. 从大到小

答案：D

13. 贪心算法每一步做局部最优选择，并希望最终得到全局最优，这依赖于什么性质？

- A. 回溯剪枝 B. 状态压缩 C. 贪心选择性质 D. 归纳假设

答案：C

14. 活动选择问题的经典贪心策略通常按活动的什么关键字排序？

- A. 结束时间从早到晚 B. 开始时间从晚到早 C. 持续时间从长到短 D. 编号随机排列

答案：A

15. 构造哈夫曼树时，每次应合并哪两个结点？

- A. 深度最大的两个 B. 权值最小的两个 C. 编号最大的两个 D. 度数最多的两个

答案：B

16. Dijkstra 算法每轮从未确定顶点中选择哪一个顶点扩展？

A. 入度最大的顶点 B. 编号最小的顶点 C. 距离最大的顶点 D. 当前距离最小的顶点

答案: D

17. 分支限界法用来舍弃不可能产生更优解分支的核心依据是什么?

A. 限界函数 B. 递归出口 C. 线性扫描 D. 稳定排序

答案: A

18. 求解最优化问题的 LC 分支限界法常用什么结构选择下一个活结点?

A. 普通队列 B. 数组下标 C. 优先队列 D. 邻接矩阵

答案: C

19. 0/1 背包分支限界法中, 常用哪种思想估计结点的价值上界?

A. 最坏情况枚举 B. 分数背包估计 C. 冒泡交换 D. 深度优先遍历

答案: B

20. 分支限界法与普通回溯法相比, 更强调利用界值做什么?

A. 保证输入有序 B. 删除所有叶结点 C. 只求任意可行解 D. 搜索并剪去无希望的最优解分支

答案: D

21. Dijkstra 算法适用于边权非负图的什么问题?

A. 最长公共子序列

B. 全排列

C. n 皇后

D. 单源最短路径

答案: D

22. 分支限界法中用于估计最优可能值的函数称为什么?

A. 哈希函数

B. 递归函数

C. 输出函数

D. 限界函数

答案：D

23. 哈夫曼树是一类带权路径长度怎样的二叉树？

- A. 固定的队列
- B. 无权数组
- C. 最小的二叉树
- D. 最大的图

答案：C

24. 贪心法每一步选择当前看来最好的方案，这称为什么？

- A. 递归出口
- B. 状态压缩
- C. 深度优先
- D. 贪心选择

答案：D

25. 回溯法通过约束条件提前放弃无效分支，这称为什么？

- A. 剪枝
- B. 归并
- C. 散列
- D. 压缩

答案：A

26. 二叉树中没有孩子的结点称为什么？

- A. 根结点
- B. 双亲结点
- C. 祖先结点
- D. 叶子结点

答案：D

27. 递推算法通常需要确定初始条件和什么？

- A. 图的颜色
- B. 队列长度
- C. 文件扩展名
- D. 递推关系

答案：D

28. 算法必须在有限步骤后结束，这体现了算法的哪一特性？

- A. 可读性
- B. 输入性
- C. 有穷性
- D. 确定性

答案：C

29. 回溯法在解空间树中通常采用哪种搜索方式？

- A. 哈希查找
- B. 深度优先搜索
- C. 广度优先搜索
- D. 顺序扫描

答案：B

30. 枚举法的核心思想是什么？

- A. 只保留局部最优
- B. 拆成独立子问题
- C. 构造状态转移表
- D. 逐一列举可能情况并检验

答案：D

31. 0/1 背包可用回溯法在解空间树中尝试“选”与什么？

- A. 编码
- B. 不选
- C. 染色
- D. 归并

答案：B

32. 分支限界法常以广度优先或最小耗费优先方式搜索什么？

- A. 解空间树
- B. 顺序表
- C. 源代码
- D. 循环队列

答案：A

33. 栈的插入和删除操作发生在哪一端？

- A. 栈底
- B. 中间
- C. 任意两端
- D. 栈顶

答案：D

34. n 皇后问题要求任意两个皇后不在同一行、同一列和同一什么位置？

- A. 权值
- B. 队列
- C. 斜线
- D. 颜色

答案：C

35. 链表插入结点时主要修改什么？

- A. 指针关系

- B. 数组容量
- C. 排序关键字
- D. 递归出口

答案：A

36. 活结点表可用队列或优先队列组织，用来决定什么？

- A. 文件路径
- B. 数组下标
- C. 扩展次序
- D. 变量类型

答案：C

37. 正在产生孩子结点的结点称为什么？

- A. 队尾结点
- B. 空结点
- C. 散列结点
- D. 扩展结点

答案：D

38. 算法每一步都有明确含义，这体现了算法的哪一特性？

- A. 最优性
- B. 确定性
- C. 有穷性
- D. 健壮性

答案：B

39. 分治算法求解后通常需要对子问题解进行什么操作？

- A. 染色
- B. 入栈

C. 合并

D. 删除

答案：C

40. 若算法只使用常数个辅助变量，其空间复杂度是多少？

A. $O(n \log n)$

B. $O(n^2)$

C. $O(1)$

D. $O(n)$

答案：C

41. 顺序表按下标访问的典型时间复杂度是多少？

A. $O(n^2)$

B. $O(1)$

C. $O(n)$

D. $O(\log n)$

答案：B

42. 矩阵连乘问题的目标是最小化什么？

A. 队列长度

B. 文件数量

C. 标量乘法次数

D. 字符个数

答案：C

43. 衡量算法随输入规模增长所需时间通常使用什么记号？

A. 大 O 记号

B. 补码

C. ASCII 码

D. 状态码

答案：A

44. 归并排序的典型时间复杂度是多少？

A. $O(n)$

B. $O(\log n)$

C. $O(2^n)$

D. $O(n \log n)$

答案：D

45. 0/1 背包动态规划常用物品编号和容量作为什么？

A. 状态

B. 颜色

C. 指针

D. 文件名

答案：A

46. 牛顿迭代法利用函数值和导数值不断逼近什么？

A. 生成树

B. 哈夫曼编码

C. 方程根

D. 最大流

答案：C

47. 二分查找要求待查找序列满足什么条件？

A. 全部相等

B. 有序

C. 无序

D. 随机

答案：B

48. 贪心算法能正确求解的问题通常具有最优子结构和什么性质？

- A. 不可分解性
- B. 文件依赖
- C. 贪心选择性质
- D. 随机性

答案：C

49. 队列的基本访问规则是什么？

- A. 优先级最高先出
- B. 先进先出
- C. 后进先出
- D. 随机访问

答案：B

50. 评价算法通常关注正确性、可读性、健壮性和什么？

- A. 效率
- B. 颜色
- C. 平台
- D. 文件名

答案：A

51. 排列树常用于描述什么问题？

- A. 散列表
- B. 全排列问题
- C. 最短路径
- D. 哈夫曼树

答案：B

52. 归并排序中合并两个有序子序列的过程称为什么？

- A. 限界
- B. 回退
- C. 归并
- D. 剪枝

答案：C

53. 最长公共子序列问题常用二维表记录什么？

- A. 磁盘地址
- B. 栈顶元素
- C. 哈希冲突
- D. 子问题最优值

答案：D

54. 分治法通常将原问题划分为若干规模较小且相互怎样的子问题？

- A. 无解的集合
- B. 随机的指针
- C. 独立的子问题
- D. 完全相同的答案

答案：C

55. 递归算法必须包含什么？

- A. 全局变量
- B. 哈希表
- C. 递归出口
- D. 随机函数

答案：C

56. 活动安排问题通常按活动结束时间进行什么操作？

- A. 排序选择
- B. 随机删除
- C. 矩阵乘法
- D. 树形编码

答案：A

57. 快速排序中确定枢轴位置的过程通常称为什么？

- A. 着色
- B. 编码
- C. 划分
- D. 装载

答案：C

58. 已产生但尚未扩展的结点称为什么？

- A. 叶子结点
- B. 死结点
- C. 根目录
- D. 活结点

答案：D

59. 动态规划适合具有最优子结构和什么特征的问题？

- A. 固定长度
- B. 重叠子问题
- C. 无序输入
- D. 随机出口

答案：B

60. 动态规划通常自底向上填写什么？

- A. 邻接点

- B. 状态表
- C. 语法树
- D. 目录树

答案: B

二、程序填空题

1. 快速排序划分。根据程序功能补全 4 处空白。

```
def partition(a, left, right):
    pivot = _____
    i, j = left, right
    _____:
        while i < j and a[j] >= pivot:
            j -= 1
        while i < j and a[i] <= pivot:
            i += 1
        _____:
            a[i], a[j] = a[j], a[i]
    a[left], a[i] = a[i], a[left]
    _____

arr = [5, 2, 7, 1, 4]
pos = partition(arr, 0, len(arr) - 1)
assert all(x <= arr[pos] for x in arr[:pos])
assert all(x >= arr[pos] for x in arr[pos + 1:])
print(arr, pos)
```

答案:

- (1) a[left]
- (2) while i < j
- (3) if i < j
- (4) return i

2. 递归求和与归纳思想。根据程序功能补全 4 处空白。

```
def sum_to(n):
    if _____:
        _____
    return _____ + _____

assert sum_to(10) == 55
print(sum_to(10))
```

答案:

- (1) n == 1
- (2) return 1

(3) `sum_to(n - 1)`

(4) `n`

3. 子集和回溯。根据程序功能补全 4 处空白。

```
def subset_sum(nums, target):
    ans, path = [], []
    def dfs(index, total):
        if _____:
            ans.append(path.copy())
            return
        if _____:
            return
        path.append(nums[index])
        dfs(index + 1, total + nums[index])
        path.pop()
    _____
    dfs(0, 0)
    _____

print(subset_sum([2, 3, 5, 7], 10))
```

答案:

(1) `total == target`

(2) `index == len(nums) or total > target`

(3) `dfs(index + 1, total)`

(4) `return ans`

4. 最长公共子序列长度。根据程序功能补全 4 处空白。

```
def lcs(x, y):
    m, n = len(x), len(y)
    dp = _____
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if _____:
                dp[i][j] = _____
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    _____

assert lcs('ABCBADAB', 'BDCABA') == 4
print(lcs('ABCBADAB', 'BDCABA'))
```

答案:

(1) `[[0] * (n + 1) for _ in range(m + 1)]`

(2) `x[i - 1] == y[j - 1]`

(3) `dp[i - 1][j - 1] + 1`

(4) `return dp[m][n]`

5. 0/1 背包分支限界。根据程序功能补全 4 处空白。

```
import heapq

def knapsack_bb(weights, values, cap):
    n = len(weights)
    best = 0
    heap = [(-sum(values), 0, 0, 0)]
    while heap:
        neg_bound, i, w, v = _____
        if _____:
            continue
        if _____:
            new_v = v + values[i]
            best = max(best, new_v)
            heapq.heappush(heap, (-(new_v + sum(values[i + 1:])), i + 1, w + weights[i],
new_v))
            heapq.heappush(heap, (-(v + sum(values[i + 1:])), i + 1, w, v))
            _____

assert knapsack_bb([2, 3, 4], [3, 4, 5], 5) == 7
print(knapsack_bb([2, 3, 4], [3, 4, 5], 5))
```

答案:

- (1) heapq.heappop(heap)
- (2) -neg_bound <= best or i == n
- (3) w + weights[i] <= cap
- (4) return best

6. n 皇后安全判断。根据程序功能补全 4 处空白。

```
def safe(cols, row, col):
    _____
    if _____ or _____:
        return False
    _____

cols = [1, 3, 0]
assert safe(cols, 3, 2) is True
print(safe(cols, 3, 2))
```

答案:

- (1) for r, c in enumerate(cols):
- (2) c == col
- (3) abs(row - r) == abs(col - c)
- (4) return True

7. 顺序求和。根据程序功能补全 4 处空白。

```
def solve(n):
    _____
```

```

    for i in _____:
        _____
    _____

```

```

assert solve(13) == 91
print(solve(13))

```

答案:

- (1) total = 0
- (2) range(1, n + 1)
- (3) total += i
- (4) return total

8. 快速排序划分思想。根据程序功能补全 4 处空白。

```

def quick_sort(a):
    _____
    return a
    _____
    left = [x for x in a[1:] if _____]
    right = [x for x in a[1:] if x > pivot]
    return _____

```

```

assert quick_sort([4, 1, 5, 2]) == [1, 2, 4, 5]
print(quick_sort([4, 1, 5, 2]))

```

答案:

- (1) if len(a) <= 1:
- (2) pivot = a[0]
- (3) x <= pivot
- (4) quick_sort(left) + [pivot] + quick_sort(right)

9. 0/1 背包。根据程序功能补全 4 处空白。

```

def knapsack(weights, values, cap):
    n = len(weights)
    dp = _____
    for i in range(1, n + 1):
        for c in range(cap + 1):
            _____
            if _____:
                dp[i][c] = max(dp[i][c], dp[i - 1][c - weights[i - 1]] + values[i - 1])
            _____

```

```

assert knapsack([2, 3, 4], [3, 4, 5], 5) == 7
print(knapsack([2, 3, 4], [3, 4, 5], 5))

```

答案:

- (1) [[0] * (cap + 1) for _ in range(n + 1)]
- (2) dp[i][c] = dp[i - 1][c]


```
(3) c >= weights[i - 1]
(4) return dp[n][cap]
```

10. 活动选择。根据程序功能补全 4 处空白。

```
def activity_select(items):
    items = _____
    ans = []

    _____
    for start, end in items:
        _____
        ans.append((start, end))

    return ans

items = [(1, 3), (2, 5), (4, 7), (6, 9)]
assert activity_select(items) == [(1, 3), (4, 7)]
print(activity_select(items))
```

答案:

```
(1) sorted(items, key=lambda x: x[1])
(2) last_end = -1
(3) if start >= last_end:
(4) last_end = end
```

11. 阶乘递归。根据程序功能补全 4 处空白。

```
def fact(n):
    _____

    _____
    return _____

assert fact(8) == 40320
_____
```

答案:

```
(1) if n <= 1:
(2) return 1
(3) n * fact(n - 1)
(4) print(fact(8))
```

12. 二分查找。根据程序功能补全 4 处空白。

```
def binary_search(a, target):
    _____
    _____

    _____
    if a[mid] == target:
        return mid
    if a[mid] < target:
```

```

        left = mid + 1
    else:
        right = mid - 1
    _____

arr = [1, 3, 5, 7, 9]
assert binary_search(arr, 7) == 3
print(binary_search(arr, 7))

```

答案:

- (1) left, right = 0, len(a) - 1
- (2) while left <= right:
- (3) mid = (left + right) // 2
- (4) return -1

13. 归并两个有序表。根据程序功能补全 4 处空白。

```

def merge(a, b):
    _____
    ans = []
    _____
    if a[i] <= b[j]:
        ans.append(a[i]); i += 1
    else:
        ans.append(b[j]); j += 1
    _____
    ans.extend(b[j:])
    _____

```

```

assert merge([1, 4, 8], [2, 3, 9]) == [1, 2, 3, 4, 8, 9]
print(merge([1, 4, 8], [2, 3, 9]))

```

答案:

- (1) i = j = 0
- (2) while i < len(a) and j < len(b):
- (3) ans.extend(a[i:])
- (4) return ans

三、综合设计题

1. 设计快速排序算法，说明一次划分如何确定基准最终位置，并完成整数序列递增排序。要求给出算法思想、关键步骤和 Python 伪代码。（10 分）

标准答案：选取首元素或末元素为基准，使用双指针把小于等于基准的元素放左侧、大于等于基准的元素放右侧，基准归位后递归处理左右子区间。伪代码：quick_sort(a, l, r)：若 l>=r 返回；p=partition(a, l, r)；quick_sort(a, l, p-1)；quick_sort(a, p+1, r)。平均时间复杂度 O(nlogn)，最坏 O(n²)，递归栈平均 O(logn)。

2. 设计分支限界算法求解 0/1 背包最大价值，要求说明上界函数、活结点选择策略和剪枝条件。（10 分）

标准答案：状态结点记录层号 i 、当前重量 w 、当前价值 v 和价值上界 $bound$ 。上界可用剩余物品的分数背包贪心估计；用优先队列按 $bound$ 从大到小取活结点。扩展“取/不取”当前物品，若重量不超过容量则更新 $best$ ；若 $bound \leq best$ 则剪枝。搜索结束返回 $best$ 。

3. 设计贪心算法求最多相容活动集合，并说明为什么应按结束时间排序。（10 分）

标准答案：将活动按结束时间非降序排序，维护 $last_end$ 表示已选活动的结束时间；依次扫描活动，若 $start \geq last_end$ 则选择该活动并更新 $last_end$ 。选择最早结束的可行活动能给后续活动留下最大剩余时间，满足贪心选择性质。时间复杂度 $O(n \log n)$ ，扫描 $O(n)$ 。

4. 设计回溯算法求给定集合中和为目标值 t 的所有子集，要求说明搜索状态和剪枝条件。（10 分）

标准答案：用 $dfs(index, total, path)$ 表示考察到第 $index$ 个元素、当前和为 $total$ 、已选元素为 $path$ 。若 $total == t$ ，记录 $path$ ；若 $index == n$ 或 $total > t$ 则返回。否则先选择 $nums[index]$ 递归，再撤销选择并递归不选。若元素非负，可用 $total > t$ 剪枝；时间复杂度最坏 $O(2^n)$ 。

5. 设计回溯算法求解 n 皇后问题，输出任意一个可行解，要求给出安全性判断函数。（10 分）

标准答案：按行放置皇后， $cols[row] = col$ 记录每行皇后列号。放置 (row, col) 前检查此前每个 (r, c) ： $c \neq col$ 且 $abs(row - r) \neq abs(col - c)$ 。若 $row == n$ 则得到解；否则枚举当前行所有列，可行则递归下一行，失败后撤销。最坏时间复杂度 $O(n!)$ 。

6. 设计二分查找算法，在升序列表中查找指定元素并返回下标，要求说明循环边界。（10 分）

标准答案：初始化 $left = 0, right = n - 1$ ；当 $left \leq right$ 时取 $mid = (left + right) // 2$ 。若 $a[mid] == target$ 返回 mid ；若 $a[mid] < target$ ，令 $left = mid + 1$ ；否则 $right = mid - 1$ 。循环结束返回 -1 。每轮搜索区间减半，时间复杂度 $O(\log n)$ ，空间复杂度 $O(1)$ 。

7. 设计快速排序主函数，对列表 $[5, 2, 7, 1, 4]$ 排序并给出测试样例和复杂度。（10 分）

标准答案：主函数调用 $quick_sort(a, 0, len(a) - 1)$ ，划分函数返回基准位置，再递归排序左右区间。测试样例 $[5, 2, 7, 1, 4]$ 输出 $[1, 2, 4, 5, 7]$ 。平均时间复杂度 $O(n \log n)$ ，最坏退化为 $O(n^2)$ ，可通过随机选基准降低退化概率。

8. 说明二分查找的循环不变式，并据此设计查找目标值的核心代码。（10 分）

标准答案：循环不变式：若目标值存在，则一定在闭区间 $[left, right]$ 中。比较 mid 后排除不可能包含目标的一半区间，并保持不变式成立。当 $left > right$ 时区间为空，目标不存在，返回 -1 。核心代码同标准二分查找，时间复杂度 $O(\log n)$ 。

9. 设计动态规划算法求 0/1 背包最大价值，要求写出状态定义、转移方程和测试样例。（10 分）

标准答案：定义 $dp[i][c]$ 为前 i 件物品、容量 c 时的最大价值。转移：不选第 i 件 $dp[i][c]=dp[i-1][c]$ ；若 $c \geq w[i-1]$ ，取 $\max(dp[i][c], dp[i-1][c-w[i-1]]+v[i-1])$ 。初始 $dp[0][c]=0$ 。样例 $weights=[2, 3, 4]$, $values=[3, 4, 5]$, $cap=5$ ，答案 7。复杂度 $O(nC)$ 。

10. 给出子集和回溯算法的 Python 主要函数，并用 $[2, 3, 5, 7]$ 和目标 10 进行说明。（10 分）

标准答案：函数 `subset_sum(nums, target)` 内部维护 `ans` 和 `path`，调用 `dfs(index, total)`。选择当前数后递归 `dfs(index+1, total+nums[index])`，再 `pop` 并递归 `dfs(index+1, total)`。样例 $[2, 3, 5, 7]$ 、 $target=10$ 可得到 $[2, 3, 5]$ 和 $[3, 7]$ 。最坏时间 $O(2^n)$ 。

11. 设计哈夫曼树构造算法，并计算给定权值集合的带权路径长度 WPL。（10 分）

标准答案：将所有权值放入小根堆；每次取出两个最小权值 x 、 y ，合并为 $x+y$ 并累加到 WPL，再把 $x+y$ 放回堆。堆中只剩一个结点时结束。每次合并最小两棵树保证 WPL 最小。时间复杂度 $O(n \log n)$ 。

12. 说明哈夫曼树贪心构造的关键步骤，并给出优先队列伪代码。（10 分）

标准答案：关键步骤：初始化小根堆；循环 $n-1$ 次，每次弹出两个最小权值，生成父结点，父权值重新入堆。伪代码：`heapify(w)`；`ans=0`；`while len(heap)>1: a=pop(); b=pop(); ans+=a+b; push(a+b)`。返回 `ans` 即 WPL。

13. 设计归并排序算法，对整数序列进行递增排序，要求说明分解与合并过程。（10 分）

标准答案：若序列长度 ≤ 1 直接返回；否则取中点分为左右两半，递归排序左右子序列，再用双指针合并两个有序序列。归并时每次取较小元素加入结果，最后追加剩余部分。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。

14. 设计递归算法计算正整数 n 的阶乘，要求说明递归出口和递归调用关系。（10 分）

标准答案：递归出口为 $n \leq 1$ 时返回 1；递归关系为 $fact(n)=n*fact(n-1)$ 。Python 伪代码：`def fact(n): if n<=1: return 1; return n*fact(n-1)`。例如 $fact(5)=120$ 。时间复杂度 $O(n)$ ，递归栈空间 $O(n)$ 。

15. 给出二分查找的 Python 函数、测试样例和时间复杂度分析。（10 分）

标准答案：函数 `binary_search(a, target)`：设置 `left`、`right`，循环 `left<=right`，计算 `mid` 并比较。样例 $a=[1, 3, 5, 7]$, $target=5$ 返回 2， $target=4$ 返回 -1。算法每次排除一半元素，时间复杂度 $O(\log n)$ ，只使用常数变量，空间复杂度 $O(1)$ 。

16. 设计 0/1 背包二维动态规划算法，要求给出完整转移表填写顺序。（10 分）

标准答案：按物品编号 i 从 1 到 n 、容量 c 从 0 到 C 填写 dp 表。先继承 $dp[i-1][c]$ ，若当前容量能放第 i 件物品，则比较选与不选的价值。答案为 $dp[n][C]$ 。填写顺序保证转移所依赖的上一行状态已经计算完成。

17. 用递归方式实现阶乘函数，并给出 $n=6$ 时的运行结果和复杂度。（10 分）

标准答案： $fact(6)=6*fact(5)$ ，逐层递归到 $fact(1)=1$ 后回溯相乘，结果为 720。主要代码：`if n<=1 return 1 else return n*fact(n-1)`。该算法进行 n 次递归调用，时间复杂度 $O(n)$ ，栈空间 $O(n)$ 。

18. 设计 n 皇后回溯算法，要求输出所有可行解而不是任意一个解。（10 分）

标准答案：在普通 n 皇后回溯基础上，当 $row==n$ 时将 $cols$ 复制加入 ans ，不立即停止搜索。每行枚举所有列，安全则放置并递归下一行，返回后撤销。安全判断仍检查同列和两条对角线。最终返回 ans ，最坏搜索复杂度 $O(n!)$ 。

19. 描述旅行商问题的判定形式，并说明其属于 NP 问题的理由。（10 分）

标准答案：判定形式：给定带权完全图和界 K ，是否存在一条经过每个城市恰好一次并回到起点、总长度不超过 K 的回路。若给出一条回路作为证书，可在多项式时间内检查是否访问所有城市一次并计算总长度是否 $\leq K$ ，因此 TSP 判定问题属于 NP。

20. 设计动态规划算法求两个字符串的最长公共子序列长度，要求写出状态转移方程。（10 分）

标准答案：定义 $dp[i][j]$ 为 x 前 i 个字符与 y 前 j 个字符的 LCS 长度。若 $x[i-1]==y[j-1]$ ， $dp[i][j]=dp[i-1][j-1]+1$ ；否则 $dp[i][j]=\max(dp[i-1][j], dp[i][j-1])$ 。初始化第 0 行第 0 列为 0，答案 $dp[m][n]$ ，复杂度 $O(mn)$ 。

21. 设计分支限界算法求简单装载问题的最优装载重量，要求说明队列式搜索过程。（10 分）

标准答案：每层决定是否装入第 i 个集装箱，结点记录当前重量 cw 和层号 i 。用队列保存活结点，扩展左子结点表示装入且 $cw+w[i]\leq C$ ，右子结点表示不装入。用剩余重量上界判断若 $cw+rest\leq best$ 则剪枝。最终 $best$ 为不超过容量的最大装载重量。

22. 说明阶乘递归算法中的状态、边界条件、返回值含义和结果输出。（10 分）

标准答案：状态为当前待求的 n ；边界条件为 $n\leq 1$ 返回 1；返回值表示 $n!$ ；递归调用 $fact(n-1)$ 求较小规模问题，再乘以 n 得到当前问题答案。输出时调用 `print(fact(n))`。例如 $n=4$ 时返回 24。

23. 设计优先队列式分支限界算法求装载问题，要求说明优先级如何确定。（10 分）

标准答案：活结点包含层号、当前重量和可能达到的最大重量上界。优先队列按上界从大到小取结点，优先扩展更可能得到最优解的分支。若结点上界不超过 best 则剪枝；可装入时生成装入分支，不装入时生成不装入分支。

24. 说明旅行商判定问题为什么是 NP 完全问题，要求包含证书验证和规约思想。（10 分）

标准答案：TSP 判定问题属于 NP，因为给定城市访问序列可多项式时间验证合法性与总距离是否不超过 K。它是 NP 困难的，因为可由已知 NP 完全问题哈密顿回路等多项式规约得到。既属于 NP 又 NP 困难，因此 TSP 判定问题是 NP 完全问题。

25. 给出归并排序的 Python 主要函数、测试样例和稳定性说明。（10 分）

标准答案：merge_sort 递归拆分数组，merge 函数用双指针合并。若 $a[i] \leq b[j]$ 时先取左侧元素，则相等关键字保持原相对次序，因此归并排序稳定。样例 [4, 1, 3, 2] 输出 [1, 2, 3, 4]。时间 $O(n \log n)$ ，空间 $O(n)$ 。

26. 设计 LCS 动态规划算法，并说明如何从 dp 表恢复一个最长公共子序列。（10 分）

标准答案：先按 LCS 转移方程填 dp 表。从 $dp[m][n]$ 开始回溯：若 $x[i-1] == y[j-1]$ ，该字符属于 LCS， $i--$ 、 $j--$ ；否则向 dp 值较大的相邻状态移动。逆序收集字符后反转，即得到一个 LCS。长度计算 $O(mn)$ ，恢复 $O(m+n)$ 。

27. 给出活动选择贪心算法的 Python 主要函数、测试样例和正确性理由。（10 分）

标准答案：先按结束时间排序，扫描并选择 $start \geq last_end$ 的活动。样例 [(1, 3), (2, 5), (4, 7), (6, 9)] 输出 [(1, 3), (4, 7)]。正确性理由是最早结束的活动不劣于其他可选活动，可通过交换论证得到最优解。

28. 说明归并排序中的状态设计、核心合并代码和最终输出结果。（10 分）

标准答案：状态是待排序子数组区间或子列表；核心合并使用 i、j 分别扫描左右有序表，较小者进入 ans，某一侧耗尽后追加另一侧剩余元素。最终输出完整有序表。分治深度 $O(\log n)$ ，每层合并总量 $O(n)$ ，总时间 $O(n \log n)$ 。